Informatics Notes

aperson31415

Now just for ORAC

Built 2026-03-29

Changed from AIO to ORAC (AIIO/FARIO) notes.

Contents

1 Helper Snippets 1

2 Strategy 2

3 Topics 3

4 Advanced Topics 5

5 Stress Testing 12

6 Miscellaneous 13

Helper Snippets (1)

boilerplate.cpp

10 lines

```
#pragma GCC optimize("O3,unroll-loops")
#include <bits/stdc++.h>
using namespace std;
#pragma GCC target("avx,avx2,bmi,bmi2,lzcnt,popcnt,sse4")

signed main() {
    cin.tie(0)->sync_with_stdio(0);
    cin.exceptions(cin.failbit);
}
```

(Most) of these functions/macros are quite short, so you should try practice typing them out, and use them in contest. However, most of them just save some typing time and improve readibility, so feel free to use none of them.

shortmacros.cpp

13 lines

```
#define len(x) ((int) (x).size())
#define elif else if
#define until(x) while(!(x))
#define all(x) (x).begin(),(x).end()
#define  in  :
#define  is  ==
#define  isnt  !=
using ll = long long;
#define _min(x, y) ((x) < (y) ? (x) : (y))
#define _max(x, y) ((x) > (y) ? (x) : (y))

int _lg(int& x) {return 31 - __builtin_clz(x);}
int _lg(ll& x) {return 63 - __builtin_clzll(x);}
```

pythonicrange.cpp

17 lines

```
struct Range {
    int start, stop, step;

    struct Iterator {
        int value, step;
        int operator*() const { return value; }
        bool operator!=(const Iterator& other) const { return value < other.value; }
        void operator++() { value += step; }
    };

    Iterator begin() const { return {start, step}; }
    Iterator end() const { return {stop, step}; }
};

inline Range range(int end) { return {0, end, 1}; }
inline Range range(int start, int end) { return {start, end, 1}; }
inline Range range(int start, int end, int step) { return {start, end, step}; }
```

inputhelpers.cpp

31 lines

```
template <typename T>
void vectorin(vector<T> &to_do) {
    for(int i = 0; i < (int) to_do.size(); i++) cin >> to_do[i];
}

template <typename T>
void input(T &thing) {cin >> thing;}

template <typename T, typename... Rest>
void input(T &first, Rest&... rest) {
    cin >> first;
    if (sizeof...(rest) > 0) input(rest...);
}

template <typename T>
void print(T &thing) {cout << thing << '\n';}

template<typename T>
void print(T thing) {cout << thing << '\n';}

template <typename T, typename... Rest>
void print(T &first, Rest&... rest) {
    if (sizeof...(rest) > 0) {cout << first << ' '; print(rest...);}
    else if (sizeof...(rest) == 0) {cout << first << '\n';}
}

template <typename T, typename... Rest>
void print(T first, Rest... rest) {
    if (sizeof...(rest) > 0) {cout << first << ' '; print(rest...);}
    else if (sizeof...(rest) == 0) {cout << first << '\n';}
}
```

fileio.cpp

2 lines

```
freopen("filein.txt", "r", stdin);
freopen("fileout.txt", "w", stdout);
```

If for some reason you forget or lose the compilation flags, here they are

compile.sh

5 lines

```
# super safe
g++ -std=c++17 sol.cpp -Og -Wall -Wextra -Wshadow -Wconversion -Wfloat-equal -
    Wduplicated-cond -Wlogical-op -Wshift-overflow=2 -Wcast-qual -Wcast-align -
    D_GLIBCXX_DEBUG -D_GLIBCXX_DEBUG_PEDANTIC -D_FORTIFY_SOURCE=2 -fsanitize=
    address -fsanitize=undefined -fno-sanitize-recover -fstack-protector -Wnull-
    dereference -Wextra-semi -Wuninitialized -Winit-self -fstack-clash-protection -
    Wduplicated-branches

# normal judge
g++ -std=c++17 sol.cpp -O2
```

Strategy (2)

Too short rn

2.1 Question

Unless the problem is super easy, solve every subtask.

Make a buncha observations and do problems by hand, on paper!

Search for techniques and alternate representations.

2.2 Competition

Solve easy problems first, then think on hard ones.

Go to toilet if you really cant think.

In the last half an hour, code some fast solves that may or may not even work.

Use all of the resources you have (cph, cppreference etc).

2.3 Debugging

Check if your logic is even correct. Check if you even read the question correctly!

Check edge cases in your logic and code, and small things like set vs multiset (set vs vector can also be differenc between tle and ac)

Topics (3)

3.1 Greedy Algorithms

Takes best choice at every step, from sorting, from edges, from middle, etc.

Most of the time, you would and should have enough time to prove these with exchange argument.

A greedy algorithm could also commonly just be a part of a bigger problem, in some q5s/6s.

3.2 Brute Force

For q1 and 2 and maybe 3, just brute force it, even if there is a way better solution.

Remember there are lots of different ways to brute force.

3.3 DP

Some are trivial DP, some are innovative, like stack von treap dude.

Remember complexity is $O(\text{base cases} + \text{states} \cdot \text{transition time})$.

Some optimisations might be useful.

3.4 Binary Search

Binary search normally is usually a part of some q4s-6s.

In c++, just use upper and lower bound for normal bsearch.

If monotonic question, very easily could be binary search on ans, if check can be in something like $O(N)$

bsearchans.cpp

6 lines

```
int lo = 0, hi = MAX + 1;
while (lo + 1 < hi) {
    int mid = (lo + hi) / 2;
    if (check(mid)) lo = mid;
    else hi = mid;
}
```

3.5 Monotone Stack

Yes very good for smaller to left/right (reverse order), also can be bigger to left/right, in amortised $O(N)$

monotonestk.cpp

12 lines

```
vector<int> closestsmallleft(vector<int> elems) {
    // Array positions, not values
    vector<int> small_toleft(elems.size());
    stack<pair<int, int>> stk;
    stk.push({0, 0});
    for (int i = 1; i <= (int) elems.size(); i++) {
        while (!stk.empty() and stk.top().first >= elems[i - 1]) stk.pop();
        small_toleft[i - 1] = stk.top().second;
        stk.push({elems[i - 1], i});
    }
    return small_toleft;
}
```

3.6 Casework

Just do everything by hand and quadriple check until ur like 'surely this isnt wrong' (for the 2nd time).

Also when implementing, can do by hand first cuz easy to edit, but check very closely.

3.7 Prefix/Suffixes

Prefix and suffix min/max is good, sums and products and xors are good.

2+ dimensioned psums is just inclusion/exclusion principle, but kinda annoying to implement cuz have to get psums of each row first.

Also part of this could be difference arrays, with 2d being even more annoying.

3.8 Pointers

2 pointers is VERY versatile amortised technique, can be used in unexpected ways.

3.9 Graphs

Most are pretty easy with adj list and some sort of algorithm. More on graphs in next section.

Some times keys will be different (i.e. 2d or strings) but algorithms will work with small adjustments.

DFS in $O(V + E)$ can find connected components well, is good enough for most graph questions, can be used in unexpected ways tho.

dfs.cpp

10 lines

```
bool visited(n, false);

void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
    for (auto u: adj[s]) {
        dfs(u);
    }
}
```

BFS in $O(V + E)$ is needed for peth length usually, a bit longer. Also 1/0 shortest path. If you can, just use DFS tho.

bfs.cpp

17 lines

```
queue<int> q;
bool visited(n, false);
int distance(n, __INT_MAX__);

visited[x] = true;
distance[x] = 0;
q.push(x);
while (!q.empty()) {
    int s = q.front(); q.pop();
    // process node s
    for (auto u : adj[s]) {
        if (visited[u]) continue;
        visited[u] = true;
        distance[u] = distance[s]+1;
        q.push(u);
    }
}
```

Flood fill in $O(V + E)$ is good for grids, but u can probably implement with dfs and bfs qucker.

floodfill.cpp

22 lines

```
const int MAX_N = 1000;

int grid[MAX_N][MAX_N];
int row_num;
int col_num;
bool visited[MAX_N][MAX_N];
int curr_size = 0;

void floodfill(int r, int c, int color) {
  if ((r < 0 || r >= row_num || c < 0 || c >= col_num)  // out of bounds
      || grid[r][c] != color
      || visited[r][c]
  ) return;

  visited[r][c] = true;
  curr_size++;

  floodfill(r, c + 1, color);
  floodfill(r, c - 1, color);
  floodfill(r - 1, c, color);
  floodfill(r + 1, c, color);
}
```

3.10 Other small ideas

Coordinate compression can be very good sometimes

Deques can be used to emulate sliding windows (or with some advanced data structures)

Learn all c++ stl cuz they can be used well

Meet in the middle

Advanced Topics (4)

4.1 Shortest Path

Bellman-Ford algorithm, shortest path in $O(VE)$ with negatives allowed, and negative cycle detection.

bellmanford.cpp

10 lines

```
for(inti=1;i<=n;i++) distance[i]=INF;

distance[x]=0;
for(int i = 1;i <= n-1; i++){
    for(auto e : edges){
        int a, b, w;
        tie(a, b, w) = e;
        distance[b] = min(distance[b], distance[a] + w);
    }
}
```

Dijkstra's algorithm, $O((V+E)\log E)$, but negatives are cooked.

dijkstra.cpp

21 lines

```
stack<int> q;
vector<int> distance;
vector<bool> processed;

for(int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0,x});

while(!q.empty()){
    int a = q.top().second; q.pop();
    if(processed[a])continue;

    processed[a] = true;
    for(auto u : adj[a]){
        int b = u.first, w = u.second;
        if(distance[a] + w < distance[b]){
        distance[b] = distance[a] + w;
        q.push({-distance[b], b});
        }
    }
}
```

Floyd-Warshall is probably not needed (cuz of $O(N^3)$), quite easy to implement.

4.2 DSU

Add edges to graph, test if vertices are connected. This code is readable, but long.

Connectivity in $O(1)$, union in $O(N)$

dsu.cpp

25 lines

```
class DisjointSets {
  private:
  vector<int> parents;
  vector<int> sizes;

  public:
  DisjointSets(int size) : parents(size), sizes(size, 1) {
    for (int i = 0; i < size; i++) { parents[i] = i; }
  }

  int find(int x) { return parents[x] == x ? x : (parents[x] = find(parents[x])); }

  bool unite(int x, int y) {
    int x_root = find(x);
    int y_root = find(y);
    if (x_root == y_root) { return false; }

    if (sizes[x_root] < sizes[y_root]) { swap(x_root, y_root); }
    sizes[x_root] += sizes[y_root];
    parents[y_root] = x_root;
    return true;
  }

  bool connected(int x, int y) { return find(x) == find(y); }
};
```

4.3 DP

Some dp snippets to speedrun dp implementation, or bash problems maybe.

LIS in $O(N\log N)$

lisdp.cpp

35 lines

```
vector<int> lis(const vector<int> &a) {
    int n = a.size();
    if (n == 0) return {};

    vector<int> tails;
    vector<int> tails_indices;
    vector<int> parent(n, -1);

    for (int i = 0; i < n; i++) {
        auto it = lower_bound(tails.begin(), tails.end(), a[i]);
        int pos = distance(tails.begin(), it);

        if (it == tails.end()) {
            tails.push_back(a[i]);
            tails_indices.push_back(i);
        } else {
            *it = a[i];
            tails_indices[pos] = i;
        }

        if (pos > 0) {
```

```
            parent[i] = tails_indices[pos - 1];
        }
    }

    vector<int> lis;
    int curr = tails_indices.back();
    while (curr != -1) {
        lis.push_back(a[curr]);
        curr = parent[curr];
    }

    reverse(lis.begin(), lis.end());
    return lis;
}
```

Not really advanced, but bitset bashing is very good, can turn 10^9 operations into $\frac{10^9}{64}$ which is around $1.5 \cdot 10^7$.

4.4 Bashing Techniques

4.4.1 Linear Bashing

Matrix expo in $(K^3 \log N)$ (cayley maclurin can be $O(K^2 \log N)$)

bm.cpp

29 lines

```
// Need bin exp function pow

vector<ll> berlekamp_massey(vector<ll> x) {
    vector<ll> ls, cur;
    ll lf, ld;
    for (int i = 0; i < x.size(); ++i) {
        ll t = 0;
        for (int j = 0; j < cur.size(); ++j)
            t = (t + x[i - j - 1] * cur[j]) % MOD;
        if ((t - x[i] + MOD) % MOD == 0) continue;
        if (!cur.size()) {
            cur.resize(i + 1);
            lf = i; ld = (t - x[i] + MOD) % MOD;
            continue;
        }
        ll k = (t - x[i] + MOD) % MOD * power(ld, MOD - 2) % MOD;
        vector<ll> c(i - lf - 1);
        c.push_back(k);
        for (int j = 0; j < ls.size(); ++j)
            c.push_back((MOD - k) * ls[j] % MOD);
        if (c.size() < cur.size()) c.resize(cur.size());
        for (int j = 0; j < cur.size(); ++j)
            c[j] = (c[j] + cur[j]) % MOD;
        if (i - lf + (int)ls.size() >= (int)cur.size())
            ls = cur, lf = i, ld = (t - x[i] + MOD) % MOD;
        cur = c;
    }
    return cur;
}
```

matrixexpo.cpp

29 lines

```
struct Matrix {
    int sz;
    vector<vector<ll>> mat;
    Matrix(int s) : sz(s), mat(s, vector<ll>(s, 0)) {}

    static Matrix identity(int s) {
        Matrix res(s);
        for (int i = 0; i < s; i++) res.mat[i][i] = 1;
        return res;
    }

    Matrix operator*(const Matrix& other) const {
        Matrix res(sz);
        for (int i = 0; i < sz; i++)
            for (int k = 0; k < sz; k++)
                for (int j = 0; j < sz; j++)
                    res.mat[i][j] = (res.mat[i][j] + mat[i][k] * other.mat[k][j]) %
                        MOD;
        return res;
    }
};

Matrix mat_pow(Matrix a, ll b) {
    Matrix res = Matrix::identity(a.sz);
    while (b > 0) {
        if (b & 1) res = res * a;
        a = a * a; b >>= 1;
    }
    return res;
}
```

guesslinrec.cpp

31 lines

```
// Needs bm snippet, and matrix snippet

ll solve_linear_recurrence(vector<ll> terms, ll n) {
    if (n < (ll)terms.size()) return terms[n];

    vector<ll> coeffs = berlekamp_massey(terms);
    int k = coeffs.size();
    if (k == 0) return 0;

    Matrix T(k);
    // Row 0: The coefficients from BM
    for (int j = 0; j < k; j++) T.mat[0][j] = coeffs[j];
    // Subsequent rows: Shifting identity to move state forward
    for (int i = 1; i < k; i++) T.mat[i][i - 1] = 1;

    // Binary Exponentiation on Matrix
    Matrix res = Matrix::identity(k);
    ll p = n - (k - 1); // We want to reach the n-th state from the (k-1)-th state
    while (p > 0) {
        if (p & 1) res = res * T;
        T = T * T;
```

```
        p >>= 1;
    }

    // Multiply by the initial values (reversed because BM returns coeffs for n-1, n
        -2...)
    ll ans = 0;
    for (int i = 0; i < k; i++) {
        ans = (ans + res.mat[0][i] * terms[k - 1 - i]) % MOD;
    }
    return ans;
}
```

NTT method in $O(K \log K \log N)$

nttlinrec.cpp

94 lines

```
typedef long long ll;
const int MOD = 998244353;
const int G = 3; // Primitive root for 998244353

void ntt(vector<int>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll wlen = power(G, (MOD - 1) / len);
        if (invert) wlen = power(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = (int)(1LL * a[i + j + len / 2] * w % MOD);
                a[i + j] = (u + v) % MOD;
                a[i + j + len / 2] = (u - v + MOD) % MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll n_inv = power(n, MOD - 2);
        for (int& x : a) x = (int)(1LL * x * n_inv % MOD);
    }
}

vector<int> multiply(vector<int> a, vector<int> b) {
    int n = 1, target = a.size() + b.size() - 1;
    while (n < target) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = (int)(1LL * a[i] * b[i] % MOD);
    ntt(a, true);
    a.resize(target);
    return a;
}

int bostan_mori(vector<int> P, vector<int> Q, ll n) {
    while (n > 0) {
        vector<int> Q_neg = Q;
        for (int i = 1; i < Q_neg.size(); i += 2) Q_neg[i] = (MOD - Q_neg[i]) % MOD;

        vector<int> U = multiply(P, Q_neg);
        vector<int> V = multiply(Q, Q_neg);

        P.clear();
        for (int i = (n & 1); i < U.size(); i += 2) P.push_back(U[i]);
        Q.clear();
        for (int i = 0; i < V.size(); i += 2) Q.push_back(V[i]);

        n >>= 1;
    }
    return (int)(1LL * P[0] * power(Q[0], MOD - 2) % MOD);
}

// a_n = x_1*a_{n-1} + x_2*a_{n-2} ... + c
int solve_recurrence_fft(vector<int> x, vector<int> a, int c, ll n) {
    int k = x.size();
    if (n < k) return a[n];

    // Calculate a[k] to handle the increased order
    ll ak = c;
    for (int i = 0; i < k; i++) ak = (ak + 1LL * x[i] * a[k - 1 - i]) % MOD;
    a.push_back((int)ak);

    // Q_old = 1 - x_1*x - x_2*x^2 ...
    vector<int> Q_old(k + 1);
    Q_old[0] = 1;
    for (int i = 1; i <= k; i++) Q_old[i] = (MOD - x[i-1]) % MOD;

    // To absorb constant c, new Q = Q_old * (1 - x)
    vector<int> Q = multiply(Q_old, {1, MOD - 1});

    // P = (a_0 + a_1*x ... + a_k*x^k) * Q mod x^{k+1}
    vector<int> P = multiply(a, Q);
    P.resize(k + 1);

    return bostan_mori(P, Q, n);
}

// Example: Fibonacci with a constant 7
// a_n = 1*a_{n-1} + 1*a_{n-2} + 7
// a_0 = 0, a_1 = 1
vector<int> x = {1, 1};
vector<int> a = {0, 1};
int c = 7;
ll n = 10; // Target index

cout << "Result: " << solve_recurrence_fft(x, a, c, n) << endl;
```

4.4.2 Polynomial Bashing

Lagrange Interpolation, in $O(N)$.

laginterpolate.cpp

33 lines

```
// Points are (0, y[0]), (1, y[1]), ..., (k, y[k])
// Find f(x)
long long lagrange(vector<long long>& y, long long x) {
    int k = y.size() - 1;
    if (x <= k) return y[x];

    vector<long long> pref(k + 1), suff(k + 1);
    pref[0] = x % MOD;
    for (int i = 1; i <= k; i++) pref[i] = pref[i - 1] * ((x - i) % MOD) % MOD;

    suff[k] = (x - k) % MOD;
    for (int i = k - 1; i >= 0; i--) suff[i] = suff[i + 1] * ((x - i) % MOD) % MOD;

    vector<long long> fact(k + 1), invFact(k + 1);
    fact[0] = 1;
    for (int i = 1; i <= k; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[k] = modInverse(fact[k]);
    for (int i = k - 1; i >= 0; i--) invFact[i] = invFact[i + 1] * (i + 1) % MOD;

    long long ans = 0;
    for (int i = 0; i <= k; i++) {
        long long num = 1;
        if (i > 0) num = num * pref[i - 1] % MOD;
        if (i < k) num = num * suff[i + 1] % MOD;

        long long den = invFact[i] * invFact[k - i] % MOD;
        if ((k - i) % 2 == 1) den = (MOD - den) % MOD;

        long long term = y[i] * num % MOD * den % MOD;
        ans = (ans + term) % MOD;
    }
    return (ans + MOD) % MOD;
}
```

4.5 Range queries

Update element in $O(1)$, update range in $O(\sqrt{N})$ (currently not implemented tho), range query in $O(\sqrt{N})$.

sqrtblocking.cpp

24 lines

```
struct Sqrt {
  int block_size;
  vector<int> nums;
  vector<long long> blocks;
  Sqrt(int sqrtn, vector<int> &arr) : block_size(sqrtn), blocks(sqrtn, 0) {
    nums = arr;
    for (int i = 0; i < nums.size(); i++) { blocks[i / block_size] += nums[i]; }
  }

  void update(int x, int v) {
    blocks[x / block_size] -= nums[x];
    nums[x] = v;
    blocks[x / block_size] += nums[x];
  }

  long long query(int r) {
    long long res = 0;
    for (int i = 0; i < r / block_size; i++) { res += blocks[i]; }
    for (int i = (r / block_size) * block_size; i < r; i++) { res += nums[i]; }
    return res;
  }

  long long query(int l, int r) { return query(r) - query(l - 1); }
};
```

Update element in $O(\log N)$, range query in $O(\log N)$, range updates not implemented.

simplebit.cpp

30 lines

```
template <class T> class BIT {
  private:
  int size;
  vector<T> bit;
  vector<T> arr;

  public:
  BIT(int size) : size(size), bit(size + 1), arr(size) {}

  void set(int ind, T val) { add(ind, val - arr[ind]); }

  void add(int ind, T val) {
    arr[ind] += val;
    ind++;
    for (; ind <= size; ind += ind & -ind) { bit[ind] += val; }
  }

  T pref_sum(int ind) {
    ind++;
    T total = 0;
    for (; ind > 0; ind -= ind & -ind) { total += bit[ind]; }
    return total;
  }

  T query(int l, int r) {
      if (l > r) return 0;
      if (l == 0) return pref_sum(r);
      return pref_sum(r) - pref_sum(l - 1);
  }
};
```

Min/max of range in $O(1)$, $O(N)$ preprocessing

sparsetable.cpp

34 lines

```
template <typename T>
```

```
class SparseTable {
private:
    int n;
    vector<vector<T>> st;

public:
    SparseTable() {}

    SparseTable(const vector<T>& arr) {
        n = arr.size();
        if (n == 0) return;

        int levels = _lg(n) + 1;
        st.assign(levels, vector<T>(n));

        for (int i = 0; i < n; i++) {
            st[0][i] = arr[i];
        }

        for (int j = 1; j < levels; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[j][i] = max(st[j - 1][i],
                               st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }

    T query(int L, int R) const {
        int len = R - L + 1;
        int j = _lg(len);
        return max(st[j][L], st[j][R - (1 << j) + 1]);
    }
};
```

Disjoint sparse table, any operation with query $O(1)$, $O(N \log N)$ preprocess.

disjointsptable.cpp

43 lines

```
template<typename T>
class DisjointSparseTable {
private:
    int n, levels;
    vector<vector<T>> table;

    // Operation: Change this to +, *, XOR, etc.
    T op(T a, T b) { return a * b; }

public:
    DisjointSparseTable(const vector<T>& arr) {
        n = arr.size();
        levels = 32 - __builtin_clz(n);
        table.assign(levels, vector<T>(n));

        for (int i = 0; i < n; i++) table[0][i] = arr[i];

        for (int h = 1; h < levels; h++) {
            int len = (1 << h);
            for (int j = 0; j < n; j += len) {
                int mid = min(n, j + len / 2);
                // Suffixes from mid-1 down to j
                table[h][mid - 1] = arr[mid - 1];
                for (int k = mid - 2; k >= j; k--)
                    table[h][k] = op(arr[k], table[h][k + 1]);

                // Prefixes from mid up to j + len
                if (mid < n) {
                    table[h][mid] = arr[mid];
                    for (int k = mid + 1; k < min(n, j + len); k++)
                        table[h][k] = op(table[h][k - 1], arr[k]);
                }
            }
        }
    }

    T query(int l, int r) {
        if (l == r) return table[0][l];
        // Find the highest bit where l and r differ
        int h = 31 - __builtin_clz(l ^ r);
        return op(table[h][l], table[h][r]);
    }
};
```

4.5.1 Segment Trees

Since seg trees are very versatile, they have their own subsection.

segtreebasic.cpp

35 lines

```
template<typename T>
struct SegmentTree {
    int n;
    vector<T> tree;
    T ID = 0;

    SegmentTree(int n) : n(n), tree(4 * n, ID) {}

    void update(int pos, T new_val) {
        update(1, 0, n - 1, pos, new_val);
    }

    T query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }

private:
    void update(int v, int tl, int tr, int pos, T new_val) {
        if (tl == tr) tree[v] = new_val;
        else {
            int tm = (tl + tr) / 2;
            if (pos <= tm) update(2 * v, tl, tm, pos, new_val);
            else update(2 * v + 1, tm + 1, tr, pos, new_val);
            tree[v] = tree[2 * v] + tree[2 * v + 1];
```

```
        }
    }

    T query(int v, int tl, int tr, int l, int r) {
        if (l > r) return ID;
        if (l == tl && r == tr) return tree[v];
        int tm = (tl + tr) / 2;
        return query(2 * v, tl, tm, l, min(r, tm)) +
               query(2 * v + 1, tm + 1, tr, max(l, tm + 1), r);
    }
};
```

segtreelazy.cpp

56 lines

```
class LazySegTree {
private:
    int n;
    vector<long long> tree, lazy;

    void apply(int v, int tl, int tr, long long val) {
        tree[v] += val * (tr - tl + 1);
        lazy[v] += val;
    }

    void push(int v, int tl, int tr) {
        if (lazy[v] != 0) {
            int tm = (tl + tr) / 2;
            apply(2 * v, tl, tm, lazy[v]);
            apply(2 * v + 1, tm + 1, tr, lazy[v]);
            lazy[v] = 0;
        }
    }

    void update(int v, int tl, int tr, int l, int r, long long add) {
        if (l > r) return;
        if (l == tl && r == tr) {
            apply(v, tl, tr, add);
        } else {
            push(v, tl, tr);
            int tm = (tl + tr) / 2;
            update(2 * v, tl, tm, l, min(r, tm), add);
            update(2 * v + 1, tm + 1, tr, max(l, tm + 1), r, add);
            tree[v] = tree[2 * v] + tree[2 * v + 1];
        }
    }

    long long query(int v, int tl, int tr, int l, int r) {
        if (l > r) return 0;
        if (l == tl && r == tr) return tree[v];
        push(v, tl, tr);
        int tm = (tl + tr) / 2;
        return query(2 * v, tl, tm, l, min(r, tm)) +
               query(2 * v + 1, tm + 1, tr, max(l, tm + 1), r);
    }

public:
    LazySegTree(int _n) : n(_n) {
        tree.assign(4 * n, 0);
        lazy.assign(4 * n, 0);
    }

    // Public API: No more tl, tr!
    void update(int l, int r, long long val) {
        update(1, 0, n - 1, l, r, val);
    }

    long long query(int l, int r) {
        return query(1, 0, n - 1, l, r);
    }
};
```

segtreepersist.cpp

66 lines

```
class PersistentSegTree {
private:
    struct Node {
        int l, r;
        long long val; // In kth-smallest problems, this represents 'count'
        Node() : l(0), r(0), val(0) {}
    };

    vector<Node> tree;
    int n;

    int update(int prev_root, int tl, int tr, int pos, int val) {
        int node = tree.size();
        tree.push_back(tree[prev_root]); // Copy old node
        if (tl == tr) {
            tree[node].val += val; // Update the leaf (increment count)
            return node;
        }
        int tm = tl + (tr - tl) / 2;
        if (pos <= tm)
            tree[node].l = update(tree[prev_root].l, tl, tm, pos, val);
        else
            tree[node].r = update(tree[prev_root].r, tm + 1, tr, pos, val);

        tree[node].val = tree[tree[node].l].val + tree[tree[node].r].val;
        return node;
    }

    long long query(int v, int tl, int tr, int l, int r) {
        if (l > r || v == 0) return 0;
        if (l == tl && r == tr) return tree[v].val;
        int tm = tl + (tr - tl) / 2;
        return query(tree[v].l, tl, tm, l, min(r, tm)) +
               query(tree[v].r, tm + 1, tr, max(l, tm + 1), r);
    }

    int find_kth(int u, int v, int tl, int tr, int k) {
        if (tl == tr) return tl;
        long long left_count = tree[tree[v].l].val - tree[tree[u].l].val;
```

```
        int tm = tl + (tr - tl) / 2;
        if (left_count >= k)
            return find_kth(tree[u].l, tree[v].l, tl, tm, k);
        else
            return find_kth(tree[u].r, tree[v].r, tm + 1, tr, k - left_count);
    }

public:
    PersistentSegTree(int _n) : n(_n) {
        tree.reserve(2000000);
        tree.push_back(Node()); // Root 0 is the "null" node
    }

    int update(int prev_root, int pos, int val) {
        return update(prev_root, 0, n - 1, pos, val);
    }

    long long query(int root_version, int l, int r) {
        return query(root_version, 0, n - 1, l, r);
    }

    int find_kth(int root_l_minus_1, int root_r, int k) {
        return find_kth(root_l_minus_1, root_r, 0, n - 1, k);
    }

    int get_initial_root() { return 0; }
};
```

segtreesparse.cpp

62 lines

```
template<typename T>
class SparseSegTree {
private:
    struct Node {
        T val;
        int l, r; // Indices to children in the 'tree' vector
        Node() : val(0), l(-1), r(-1) {}
    };

    vector<Node> tree;
    long long L, R; // Range of the segment tree (e.g., 0 to 1e9)

    // Internal function to create a new node and return its index
    int newNode() {
        tree.emplace_back();
        return (int)tree.size() - 1;
    }

public:
    // Constructor defines the coordinate bounds
    SparseSegTree(long long min_coord, long long max_coord) : L(min_coord), R(
        max_coord) {
        tree.reserve(200000); // Pre-reserve to avoid frequent reallocations
        newNode(); // Create root at index 0
    }

    void update(long long pos, T delta) {
        update(0, L, R, pos, delta);
    }

    T query(long long ql, long long qr) {
        return query(0, L, R, ql, qr);
    }

private:
    void update(int v, long long tl, long long tr, long long pos, T delta) {
        if (tl == tr) {
            tree[v].val += delta;
            return;
        }
        long long tm = tl + (tr - tl) / 2;
        if (pos <= tm) {
            if (tree[v].l == -1) tree[v].l = newNode();
            update(tree[v].l, tl, tm, pos, delta);
        } else {
            if (tree[v].r == -1) tree[v].r = newNode();
            update(tree[v].r, tm + 1, tr, pos, delta);
        }

        T left_val = (tree[v].l != -1) ? tree[tree[v].l].val : 0;
        T right_val = (tree[v].r != -1) ? tree[tree[v].r].val : 0;
        tree[v].val = left_val + right_val;
    }

    T query(int v, long long tl, long long tr, long long ql, long long qr) {
        if (v == -1 || ql > tr || qr < tl) return 0;
        if (ql <= tl && tr <= qr) return tree[v].val;

        long long tm = tl + (tr - tl) / 2;
        return query(tree[v].l, tl, tm, ql, qr) +
               query(tree[v].r, tm + 1, tr, ql, qr);
    }
};
```

4.6 Trees

eulertour.cpp

7 lines

```
void euler_tour(int at, int prev) {
  start[at] = timer++;
  for (int n : neighbors[at]) {
    if (n != prev) { euler_tour(n, at); }
  }
  end[at] = timer;
}
```

MST in $O(E \log E)$.

kruskal.cpp

9 lines

```
pair<int, int> edges[MAX_E];
// order edges by weight ascending
```

```
for (int i = 1; i <= V; i++) ms(i);
for (int i = 0; i < E; i++) {
    if (f(edges[i].first) != f(edges[i].second)) {
        // Add edges[i] to minumum spanning tree
        u(edges[i].first, edges[i].second);
    }
}
```

TopoSort in $O((V+E)\log E)$ i think.

kahn.cpp

16 lines

```
vector<int> graph[MAX_V], S, top_sort;
int deps[MAX_V];
for (int i = 1; i <= V; i++) {
    for (int j = 0; j < graph[i].size(); j++) {
        deps[graph[i][j]]++;
    }
}
for (int i = 1; i <= V: i++) if (deps[i] == 0) S.push_back(i);
while (!S.empty()) {
    int u = s.back(); s.pop_back();
      top_sort.push_back(u);
    for (int i = 0; i < graph[u].size(); i++) {
        if (--deps[graph[u][i]] == 0) S.push_back(graph[u][i]);
    }
}
if (top_sort.size() != V) assert(false && "Graph contains cycle");
```

LCA in $O(1)$, $O(N\log N)$ preprocessing.

lca.cpp

58 lines

```
class LCA {
private:
    int n, m;
    vector<int> first_visit, tour, depths, lg;
    vector<vector<int>> st;

    void dfs(const vector<vector<int>>& adj, int u, int p, int d) {
        first_visit[u] = tour.size();
        tour.push_back(u);
        depths.push_back(d);
        for (int v : adj[u]) {
            if (v != p) {
                dfs(adj, v, u, d + 1);
                tour.push_back(u);
                depths.push_back(d);
            }
        }
    }

public:
    LCA(const vector<vector<int>>& adj, int root = 0) {
        n = adj.size();
        first_visit.resize(n);
        tour.reserve(2 * n);
        depths.reserve(2 * n);

        dfs(adj, root, -1, 0);

        m = tour.size();
        int max_log = 31 - __builtin_clz(m) + 1;
        st.assign(max_log, vector<int>(m));
        lg.assign(m + 1, 0);

        for (int i = 2; i <= m; i++) lg[i] = lg[i / 2] + 1;
        for (int i = 0; i < m; i++) st[0][i] = i;

        for (int j = 1; j < max_log; j++) {
            for (int i = 0; i + (1 << j) <= m; i++) {
                int a = st[j - 1][i];
                int b = st[j - 1][i + (1 << (j - 1))];
                st[j][i] = (depths[a] < depths[b]) ? a : b;
            }
        }
    }

    int get_lca(int u, int v) {
        int L = first_visit[u], R = first_visit[v];
        if (L > R) swap(L, R);
        int j = lg[R - L + 1];
        int a = st[j][L];
        int b = st[j][R - (1 << j) + 1];
        return tour[(depths[a] < depths[b]) ? a : b];
    }

    int get_dist(int u, int v) {
        return depths[first_visit[u]] + depths[first_visit[v]] - 2 * depths[
            first_visit[get_lca(u, v)]];
    }
};
```

Stress Testing (5)

You should only EVER use this in one case, as a last resort (takes so long to implement and check), if the servers or school internet fails. But if you are cooked so badly that uve done everything you can, and went to the toilet etc, u can use this (why would you be).

tester.sh

45 lines

```bash
#!/bin/bash

# --- Configuration ---
TIME_LIMIT="2s"
NUM_TESTS=100
# ---------------------

echo "Compiling files..."
g++ sol.cpp -std=c++17 -O2 -o solution || exit 1
g++ -O3 gen.cpp -o gen || exit 1
g++ -O3 checker.cpp -o checker || exit 1

for ((i = 1; i <= NUM_TESTS; ++i)); do
    # 1. Generate test case
    ./gen $i > input.txt

    # 2. Run solution with timeout
    timeout $TIME_LIMIT ./solution < input.txt > out.txt
    exit_code=$?

    # 3. Handle Runtime Errors or Timeouts
    if [ $exit_code -eq 124 ]; then
        echo "TLE $i (> $TIME_LIMIT)"
        exit 1
    elif [ $exit_code -ne 0 ]; then
        echo "RE $i (Exit Code $exit_code)"
        echo "--- Input ---"
        cat input.txt
        exit 1
    fi

    # The checker reads input.txt and out.txt to verify validity
    if ! ./checker; then
        echo "WA $i"
        echo "--- Input ---"
        cat input.txt
        echo "--- Your Output ---"
        cat out.txt
        exit 1
    fi

    echo "AC $i"
done

echo "Successfully passed $NUM_TESTS tests!"
```

Here are some sample snippets for the checker and generator files:

gen.cpp

14 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

// Helper to get random numbers in range [l, r]
long long rng(long long l, long long r, mt19937& gen) {
    uniform_int_distribution<long long> dist(l, r);
    return dist(gen);
}

int main(int argc, char* argv[]) {
    // Seed is from bash file, u don't have to use tho
    int seed = atoi(argv[1]);
    mt19937 gen(seed);
}
```

checker.cpp

19 lines

```cpp
#include <bits/stdc++.h>
using namespace std;

/**
 * EXIT CODES:
 * 0: AC (Accepted)
 * 1: WA (Wrong Answer / Logic Error)
 */

int main() {
    ifstream fin("input.txt");    // The problem input
    ifstream fout("out.txt");     // Your solution's output

    // Use fin and fout to get input, make sure format is correct first.
    // If anything goes wrong, return 1.

    // If we reached here, everything is correct
    return 0;
}
```

Miscellaneous (6)

6.1 Binary Search Variants

Ternary Search: Used to find min/max of unimodal function

ternsearch.cpp

14 lines

```
double ternary_search(double l, double r) {
    //set error limit
    double eps = 1e-9;
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);
        double f2 = f(m2);
        if (f1 < f2) l = m1;
        else r = m2;
    }
    //return the maximum of f(x) in [l, r]
    return f(l);
}
```

Binary Search on psuedo-derivative (forward differences): Same application as ternary search.

bsearchdx.cpp

10 lines

```
double low = 0, high = 1e9;
double h = 1e-9; // Your delta

for(int i = 0; i < 100; i++) {
    double mid = low + (high - low) / 2.0;

    if (f(mid + h) > f(mid)) high = mid;
    else low = mid;
}
double min_x = low;
```

6.2 Minimum-storing data structures

Just a queue with more constant factor, but can get min/max element in $O(1)$

minqueue.cpp

23 lines

```
template <typename T>
struct MinQueue {
    deque<T> q;       // Stores all elements currently in the queue
    deque<T> min_q;   // Stores elements in non-decreasing order

    void push(T val) {
        q.push_back(val);
        while (!min_q.empty() && min_q.back() > val) {
            min_q.pop_back();
        }
        min_q.push_back(val);
    }

    void pop() {
        if (q.empty()) return;
        if (q.front() == min_q.front()) min_q.pop_front();
        q.pop_front();
    }

    T get_min() return min_q.front();
    bool empty() return q.empty();
    int size() return q.size();
};
```

Just a stack with more constant factor, can get min/max in $O(1)$

minstack.cpp

19 lines

```
template <typename T>
struct MinStack {
    vector<pair<T, T>> s;

    void push(T val) {
        if (s.empty()) {
            s.push_back({val, val});
        } else {
            T current_min = min(val, s.back().second);
            s.push_back({val, current_min});
        }
    }

    void pop() if (!s.empty()) s.pop_back();
    T top() return s.back().first;
    T get_min() return s.back().second;
    bool empty() return s.empty();
    int size() return s.size();
};
```

6.3 Order Statistic Tree

orderedset.cpp

9 lines

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> prdered_set;

// Functionality identical to std::set except for 2 added methods:
// find_by_order(k): returns iterator to element at index k
// order_of_key(n): returns number of items strictly less than n
```

orderedmultiset.cpp

49 lines

```
template <typename T>
class ordered_multiset {
  private:
    // Pair: (value, unique_id)
    typedef pair<T, long long> TPair;
    struct cmp {
        bool operator()(const TPair &a, const TPair &b) const {
            if (a.first != b.first) return a.first < b.first;
            return a.second < b.second; // tie-break by unique ID
        }
    };

    typedef tree<
        TPair,
        null_type,
        cmp,
        rb_tree_tag,
        tree_order_statistics_node_update
    > ost_type;

    ost_type ost;
    long long uid = 0; // automatic ID counter

  public:
    void insert(const T &value) ost.insert({value, uid++});

    // Erase one occurrence
    bool erase(const T &value) {
        auto it = ost.lower_bound({value, LLONG_MIN});
        if (it != ost.end() && it->first == value) {
            ost.erase(it);
            return true;
        }
        return false;
    }

    // Number of elements strictly less than value
    size_t order_of_key(const T &value) const return ost.order_of_key({value,
        LLONG_MIN});

    // k-th smallest element (0-indexed)
    optional<T> find_by_order(size_t k) const {
        if (k >= ost.size()) return nullopt;
        return ost.find_by_order(k)->first;
    }

    size_t size() const return ost.size();

    bool empty() const return ost.empty();
};
```

orderedmap.cpp

8 lines

```
template <typename Key, typename Value>
using ordered_map = tree<
    Key,
    Value,
    less<Key>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;
```

6.4 Mathematics

A bunch of geometry implementations and combi/nt.

lcmgcd.cpp

2 lines

```
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
int lcm(int a, int b) { return a*b / gcd(a, b); }
```

binexp.cpp

11 lines

```
ll exp(ll x, ll n, ll m) {
  assert(n >= 0);
  x %= m;  // note: m * m must be less than 2^63 to avoid ll overflow (mod is m)
  ll res = 1;
  while (n > 0) {
    if (n % 2 == 1) { res = res * x % m; }
    x = x * x % m;
    n /= 2;
  }
  return res;
}
```

modularint.cpp

27 lines

```
struct mi {
  int v;
  explicit operator int() const { return v; }
  mi() { v = 0; }
  mi(long long _v) : v(_v % MOD) { v += (v < 0) * MOD; }
};
mi &operator+=(mi &a, mi b) {
  if ((a.v += b.v) >= MOD) a.v -= MOD;
  return a;
}
mi &operator-=(mi &a, mi b) {
  if ((a.v -= b.v) < 0) a.v += MOD;
  return a;
}
mi operator+(mi a, mi b) { return a += b; }
mi operator-(mi a, mi b) { return a -= b; }
mi operator*(mi a, mi b) { return mi((long long)a.v * b.v); }
mi &operator*=(mi &a, mi b) { return a = a * b; }
mi pow(mi a, long long p) {
  assert(p >= 0);
  return p == 0 ? 1 : pow(a * a, p / 2) * (p & 1 ? a : 1);
}
mi inv(mi a) {
  assert(a.v != 0);
```

```
  return pow(a, MOD - 2);
}
mi operator/(mi a, mi b) { return a * inv(b); }
```

If u don't want to fully write out the mod int class, but need division.

modinverse.cpp

16 lines

```
// Only works if M is prime, uses lls and bin exp

long long power(long long base, long long exp) {
    long long res = 1;
    base %= MOD; // Use your M here
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp /= 2;
    }
    return res;
}

long long modInverse(long long n, long long MOD = 1000000007) {
    return power(n, M - 2);
}
```

rectinterset.cpp

8 lines

```
struct Rect {int x1, y1, x2, y2;};

int intersect(Rect p, Rect q) {
  // Calculate overlap in x and y directions
  int xOverlap = max(0, min(p.x2, q.x2) - max(p.x1, q.x1));
  int yOverlap = max(0, min(p.y2, q.y2) - max(p.y1, q.y1));
  return xOverlap * yOverlap;  // Area of intersection
}
```

polyarea.cpp

11 lines

```
struct point {double x, y;};

double area(const vector<point>& fig) {
    double res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        point p = i ? fig[i - 1] : fig.back();
        point q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return fabs(res) / 2;
}
```

A bunch of tricks for vectors & points.

point.cpp

27 lines

```
struct Point {
    ll x, y;
    Point operator+(const Point& o) const { return {x + o.x, y + o.y}; }
    Point operator-(const Point& o) const { return {x - o.x, y - o.y}; }
    bool operator==(const Point& o) const { return x == o.x && y == o.y; }

    ll dot(const Point& o) const { return x * o.x + y * o.y; }
    ll cross(const Point& o) const { return x * o.y - y * o.x; }

    // Use for: Comparing distances without using sqrt()
    ll distSq(const Point& o) const {
        return (x - o.x) * (x - o.x) + (y - o.y) * (y - o.y);
    }

    // 1 (Left), -1 (Right), 0 (Collinear)
    int cross_prod_sign(Point b, Point c) const {
        ll val = (b - *this).cross(c - *this);
        if (val == 0) return 0;
        return (val > 0) ? 1 : -1;
    }

    // Lies on AB
    bool onSegment(Point a, Point b) const {
        return cross_prod_sign(a, b) == 0 && (a - *this).dot(b - *this) <= 0;
    }
};
```

6.5 C++ Tricks

cpptricks.cpp

15 lines

```
// Cool trick: if a is b, cout << " " else, cout "\n". Useful for graphs
cout << " \n"[a == b];

// Custom literals!
long long operator "" _km(unsigned long long literal) {return literal * 1000};
8_km // returns 8000

// If x is != 0, returns 1, else 0
!!x

// Initialisers in ifs, works with whiles
if(int x = f(); is_good(x)) //

// Commas in fors, also works with ifs, whiles etc
for(int a = 0, int b = 0; a != b; a++) //
```

6.6 Topic Searching

Here's a big list of topics/patterns to search for.

topics.txt

17 lines

```
Binary Search on ans
Binary Search
Greedy
Monotone Stack
Prefix Sums
```

```
Sliding Window
2 Pointers
DP
Interval
Scheduling
Brute Force
Difference Array
Interval Merging with greedy sorting
Greedy sorting and binary search
Shortest path
Bashing with bm & matrices (or fft)
... i need to add more, maybe as i solve problems
```